



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA

Tarea 11
Bases de Datos

P R E S E N T A

Luis Alejandro López Márquez

P R O F E S O R

Fernando Arreola Franco

Grupo 1

Fecha de entrega: 03/05/2026



Ciudad Universitaria, Cd. Mx., 2026

1. Definición

Una subconsulta (también llamada subquery, consulta anidada o consulta interna) es una sentencia SELECT que se encuentra embebida dentro de otra sentencia SQL, denominada consulta externa o principal. La consulta interna se ejecuta primero y su resultado es utilizado como entrada por la consulta externa para completar la operación.

Formalmente, una subconsulta es cualquier consulta que aparece dentro de otra consulta encerrada entre paréntesis. Pueden devolver un único valor escalar, una sola fila, una sola columna o una tabla completa, según el contexto en que se empleen.

La idea fundamental es que permiten descomponer problemas complejos en partes más simples y legibles, evitando en muchos casos la necesidad de crear tablas temporales o vistas intermedias.

2. ¿Dónde puedo usarlas y bajo qué condiciones?

Las subconsultas pueden aparecer en distintas cláusulas de una sentencia SQL, siempre que el contexto acepte una expresión o una tabla. Las ubicaciones más comunes son:

2.1 En la cláusula WHERE

Es el uso más frecuente. La subconsulta filtra filas de la consulta externa comparando un atributo con el resultado de la consulta interna. Las condiciones habituales son:

- Comparadores escalares (=, >, <, >=, <=, !=): la subconsulta debe devolver exactamente un valor.
- IN / NOT IN: la subconsulta devuelve una columna con múltiples valores.
- EXISTS / NOT EXISTS: evalúa si la subconsulta devuelve al menos una fila.
- ANY / ALL: compara el valor con cualquiera o con todos los valores del resultado.

2.2 En la cláusula FROM (subconsulta como tabla derivada)

La subconsulta se trata como una tabla temporal y debe llevar un alias. Resulta útil cuando se necesita agrupar o filtrar antes de hacer un JOIN con otra tabla.

```
SELECT dep.departamento, dep.promedio
FROM (
    SELECT departamento, AVG(sueldo) AS promedio
```

```

FROM empleado

GROUP BY departamento

) AS dep

WHERE dep.promedio > 10000;

```

2.3 En la cláusula **SELECT** (subconsulta escalar)

Se coloca dentro de la lista de proyección y debe devolver exactamente un valor por cada fila procesada. Permite calcular un valor derivado junto con cada registro de la consulta principal.

```

SELECT nombre,

        sueldo,

        (SELECT AVG(sueldo) FROM empleado) AS sueldo_promedio

FROM empleado;

```

2.4 En la cláusula **HAVING**

Se usa para filtrar grupos después de aplicar GROUP BY, cuando la condición involucra una función de agregación calculada en otra consulta.

```

SELECT departamento, COUNT(*) AS total

FROM empleado

GROUP BY departamento

HAVING COUNT(*) > (

    SELECT AVG(cnt) FROM (

        SELECT COUNT(*) AS cnt FROM empleado GROUP BY departamento

    ) AS sub

);

```

Condiciones de uso

Para emplear correctamente una subconsulta se deben cumplir las siguientes condiciones:

- Cardinalidad adecuada: al usar = o un comparador escalar, la subconsulta debe devolver una sola fila y una sola columna. Si devuelve más de una fila, el motor lanza un error.

- **Compatibilidad de tipos:** el tipo de dato devuelto por la subconsulta debe ser compatible con el atributo con el que se compara en la consulta externa.
- **Referencia correcta de atributos:** los atributos referenciados dentro de la subconsulta deben pertenecer a su propio ámbito o a la consulta externa (en el caso de subconsultas correlacionadas).
- **Rendimiento:** las subconsultas no correlacionadas se ejecutan una sola vez, por lo que son más eficientes. Las correlacionadas se ejecutan una vez por cada fila de la consulta externa.

3. Subconsultas Correlacionadas

Una subconsulta correlacionada es aquella que hace referencia a una o más columnas de la tabla o alias definido en la consulta externa. A diferencia de una subconsulta simple (no correlacionada), cuyo resultado es independiente y se calcula una sola vez, la subconsulta correlacionada se evalúa una vez por cada fila procesada por la consulta externa, creando una dependencia entre ambas consultas.

Diferencia con subconsultas no correlacionadas

- **Subconsulta no correlacionada:** se ejecuta de forma autónoma, primero se resuelve la interna y luego se usa su resultado en la externa.
- **Subconsulta correlacionada:** la consulta interna depende de un valor de la fila actual de la consulta externa; se re-evalúa en cada iteración.

Sintaxis general

```
SELECT col_externa
FROM tabla_externa t_ext
WHERE valor_externo operador (
    SELECT funcion(col_interna)
    FROM tabla_interna t_int
    WHERE t_int.atributo = t_ext.atributo    -- referencia a la tabla externa
);
```

La clave está en la última línea del ejemplo: la condición WHERE de la subconsulta usa `t_ext.atributo`, que pertenece a la tabla de la consulta principal. Esto crea la correlación.

Uso con EXISTS

Una de las aplicaciones más habituales de las subconsultas correlacionadas es junto con EXISTS, para verificar la existencia de registros relacionados sin necesidad de JOIN:

Empleados que tienen al menos un préstamo registrado

```
SELECT nombre
FROM cliente c
WHERE EXISTS (
    SELECT 1
    FROM prestatario p
    WHERE p.nombreCliente = c.nombreCliente
);
```

Consideraciones de rendimiento

Dado que una subconsulta correlacionada se ejecuta N veces (una por fila de la consulta externa), puede resultar costosa en tablas grandes. En muchos casos, puede ser reescrita como un JOIN para mejorar el rendimiento:

```
--Equivalente con JOIN (más eficiente)
SELECT DISTINCT c.nombre
FROM cliente c
INNER JOIN prestatario p ON p.nombreCliente = c.nombreCliente;
```

4. Ejemplos

A continuación se presentan ejemplos concretos usando el esquema bancario del curso, formado por las tablas: cuenta, sucursal, cliente, ctaCliente, prestamo y prestatario.

Ejemplo 1 — Subconsulta escalar en WHERE con =

Obtener el nombre de los clientes que viven en la misma ciudad que la sucursal 'Copilco'.

```
SELECT nombreCliente
FROM cliente
WHERE ciudad = (
```

```
SELECT ciudad
FROM sucursal
WHERE nombreSucursal = 'copilco'
);
```

La subconsulta devuelve un único valor (la ciudad de la sucursal Copilco) y la consulta externa filtra los clientes que viven en esa misma ciudad.

Ejemplo 2 — Subconsulta con IN

Listar los nombres de todos los clientes que tienen al menos una cuenta.

```
SELECT nombreCliente
FROM cliente
WHERE nombreCliente IN (
    SELECT nombreCliente
    FROM ctaCliente
);
```

La subconsulta devuelve el conjunto de nombres de clientes con cuenta; la consulta externa selecciona solo aquellos presentes en ese conjunto.

Ejemplo 3 — Subconsulta con NOT IN (diferencia)

Obtener los nombres de clientes que tienen cuenta pero no tienen préstamo.

```
SELECT nombreCliente
FROM ctaCliente
WHERE nombreCliente NOT IN (
    SELECT nombreCliente
    FROM prestatario
);
```

Ejemplo 4 — Subconsulta en FROM (tabla derivada)

Encontrar las sucursales cuyo saldo promedio de cuentas supera el promedio general de todos los saldos.

```
SELECT nombreSucursal, promedio_sucursal
```

```

FROM (
    SELECT nombreSucursal, AVG(saldo) AS promedio_sucursal
    FROM cuenta
    GROUP BY nombreSucursal
) AS promedios
WHERE promedio_sucursal > (
    SELECT AVG(saldo) FROM cuenta
);

```

Ejemplo 5 — Subconsulta correlacionada con EXISTS

Mostrar los nombres de clientes que tienen préstamo con importe mayor a 5,000 pesos.

```

SELECT c.nombreCliente
FROM cliente c
WHERE EXISTS (
    SELECT 1
    FROM prestatario pr
    INNER JOIN prestamo p ON pr.numPrestamo = p.numPrestamo
    WHERE pr.nombreCliente = c.nombreCliente  -- correlación
    AND p.importe > 5000
);

```

Para cada fila de cliente, el motor evalúa si existe al menos un préstamo de ese cliente con importe mayor a 5,000. La referencia c.nombreCliente dentro de la subconsulta es la que la convierte en correlacionada.

Ejemplo 6 — Subconsulta con ALL

Obtener el nombre de la sucursal con los activos más altos (mayor que todos los demás).

```

SELECT nombreSucursal
FROM sucursal
WHERE activos >= ALL (

```

```
SELECT activos  
  
FROM sucursal  
  
);
```

Referencias

- PostgreSQL Global Development Group. (2024). Subquery Expressions. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/current/functions-subquery.html>
- GeeksforGeeks. (2024). SQL Subquery. <https://www.geeksforgeeks.org/sql-subquery/>